

- 1 Please describe the advantage and disadvantage of **inline function**. (5%) What kind of functions are suitable to be written as **inline functions**? (5%)
2. Write a simple C++ program to demonstrate "**function overloading**". You should also explain your program. (10%)
3. Explain what the difference between the **smart pointer** and the **raw pointer** is. (10%)
4. Explain what **abstract classes** are, and write an abstract class in C++. (10%)
5. Explain what **higher-order functions** are, and write a higher-order function in C++. (10%)
6. Write a simple C++ program to demonstrate the use of **Lambda Expression**. You should also explain your program. (10%)
- 7 What is the output of the following program? (10%)

```
#include<iostream>
using namespace std;
class Shape {
public:
    double area;
    virtual double calculateArea() { return 0.0; };
    void printArea() { cout << "(Shape) " << area << endl; }
};

class Square: public Shape {
public:
    Square(double s) {side =s;}
    double side;
    double calculateArea() { return side * side; };
    void printArea() { cout << "(Square) " << area << endl; }
};

void f1(Shape s) {
    s.area = s.calculateArea();
    s.printArea();
}

void f2(Shape *s) {
    s->area = s->calculateArea();
    s->printArea();
}

int main(int argc, const char * argv[]) {
    Square square(5.0);
    f1(square);
    f2(&square);
    square.printArea();
    Shape *sp = new Square(3.0);
    f1(*sp);
    f2(sp);
    sp->printArea();
}
```

8. What is the output of the following program? (10%)

```
#include<iostream>
using namespace std;
class TwoDoubles {
public:
    double x; double y;
    TwoDoubles(double x, double y) {this->x = x+1.1; this->y = y;}
    operator int()const { return x + y; }
    operator double() const { return (int)x - y; }
};

int main(int argc, const char * argv[]) {
    TwoDoubles td(6.3, 1.2);
    cout << (int)td.x << endl;          // (8.1) (2%)
    cout << (double)td.y << endl;       // (8.2) (2%)
    cout << (int)td << endl;            // (8.3) (3%)
    cout << (double)td << endl;         // (8.4) (3%)
}
```

9. What is the output of the following program? (10%)

```
#include<iostream>
using namespace std;
template<class T, class S> class EqualChecker {
    public: bool operator()(T d1, S d2) {return (d1==(S)d2)? true:false;}
};
int main() {
    cout << boolalpha;
    int i1 = 6, i2 = 8, i3 = 6;
    double d1 = 6.3, d2 = 8.6;
    EqualChecker<int, double> equalchecker0;
    EqualChecker<double, int> equalchecker1;
    EqualChecker<int, int> equalchecker2;
    EqualChecker<int *, int *> equalchecker3;
    cout << equalchecker0(6, 6.3) << endl; // (9.1) (2%)
    cout << equalchecker0(8.6, 8) << endl; // (9.2) (2%)
    cout << equalchecker1(6, 6.3) << endl; // (9.3) (2%)
    cout << equalchecker1(8.6, 8) << endl; // (9.4) (2%)
    cout << equalchecker2(6, 6) << endl; // (9.5) (1%)
    cout << equalchecker3(&i1, &i3) << endl; // (9.6) (1%)
}
```

10. (1) Figure out the lines that will cause compilation error. (2) After removing the lines that will cause compilation error, what is the output of this program? (10%)

```
01 #include<iostream>
02 using namespace std;
03 class A {
04 private:
05     int a1;
06 protected:
07     int a2;
08 public:
09     int a3;
10     A(int value) { a1 = value+1; a2 = value+2; a3 = value+3; }
11     A(const A &a) { a1 = a.a1+3; a2 = 2; }
12     int fun1() { return (a1); }
13 };
14 class B: public A {
15 private:
16     int b1;
17 protected:
18     int b2;
19 public:
20     B(): A(6) { b1 = 3; b2 = 5; }
21     int fun1() { return A::fun1(); }
22     int fun2() { return (b1+a1); }
23     int fun3() { return (b2+a2); }
24 };
25 int main(int argc, const char * argv[]) {
26     A a = A(8);
27     B b1;
28     B b2 = b1;
29     cout << "(1): " << a.a3 << endl;
30     cout << "(2): " << b1.a2 << endl;
31     cout << "(3): " << b1.a3 << endl;
32     cout << "(4): " << a.fun1() << endl;
33     cout << "(5): " << b1.fun1() << endl;
34     cout << "(6): " << b2.fun1() << endl;
35     cout << "(7): " << a.fun3() << endl;
36     cout << "(8): " << b1.fun3() << endl;
37     cout << "(9): " << b2.fun3() << endl;
38 }
```